UniDebugger-Local: A Cost-Efficient Hierarchical Multi-Agent Framework for Software Debugging

Stanley Chung
Department of Computer Science
Columbia University
New York, NY
Email: syc2153@columbia.edu

***Abstract*—Software debugging remains a labor-intensive bottleneck in CI/CD pipelines. While Large Language Models (LLMs) show potential, isolated prompting often fails for complex, repo-level bugs. This project proposes *UniDebugger-Local*, a rebuild of the UniDebugger framework. By utilizing a 3-level hierarchical coordination of 6 specialized agents, this framework mimics human cognitive processes. To optimize for cost, this version replaces the external search-based Helper agent with local static analysis and documentation retrieval, ensuring the only variable cost is the LLM API.**

I. Problem Statement and Motivation

Traditional Automated Program Repair (APR) methods often focus on isolated steps like Fault Localization (FL) or Patch Generation. Existing multi-agent systems typically use mesh-like communication, which is computationally expensive and prone to "context explosion" on trivial bugs.

The original UniDebugger addressed this via a hierarchical model. However, for a student-led implementation, the reliance on paid search APIs (e.g., Tavily) for the "Helper" agent presents a financial barrier. There is a need for a version that achieves state-of-the-art results on benchmarks like Defects4J using only local repository context and open-source analysis tools.

II. Proposed Approach: UniDebugger-Local

We propose a 6-agent system organized into three progressive levels of intensity.

A. Hierarchical Coordination Architecture

- **Level 1 (L1):** Quick repair for simple logic errors using **Locator** and **Fixer** agents.
- **Level 2 (L2):** Triggered upon L1 failure. Adds **Slicer** and **Summarizer** to handle single-file complexity.
- **Level 3 (L3):** Triggered for cross-file bugs. Activates **RepoFocus** to identify dependencies.

B. The 6-Agent Specialization

To maintain performance without the paid Helper agent, we redistribute its responsibilities:

1) **RepoFocus:** Analyzes cross-file dependencies to provide a list of 2–6 bug-related files.
2) **Summarizer:** Generates concise function-level summaries to align code with LLM training objectives.
3) **Slicer:** Isolates 20–100 lines of suspicious code using **SonarQube Community Edition**.
4) **Locator:** Annotates faulty lines with `// buggy line` comments using execution feedback.
5) **Fixer:** Generates a git-diff patch based on marked code.
6) **FixerPro:** Evaluates patches against test results (JUnit) and refactors for simplicity.

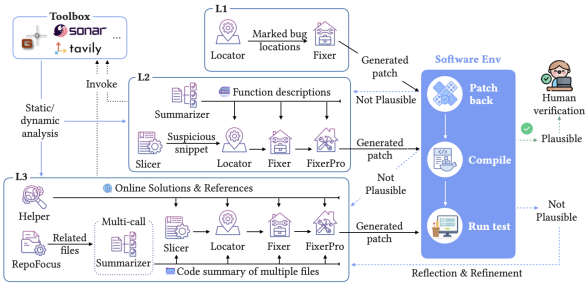

Fig. 1. System Architecture

III. Cost-Efficient Tool Integration

To ensure the only cost is the LLM API (GPT-4o/Gemini 1.5 Pro), we integrate:

- **Static Analysis:** *SonarQube Community Edition* (Free) for Slicer.
- **Fault Localization:** *GZoltar* (Free/Open-Source) for the Locator agent.
- **Search Replacement:** Local documentation RAG using *Tree-sitter* for internal symbol mapping.

IV. Evaluation Plan

We will evaluate *UniDebugger-Local* on the **QuixBugs** benchmark.

- **Metrics:** We will measure the number of Plausible fixes (passing all tests) and Correct fixes (manually verified).
- **Baselines:** Performance will be compared against the original UniDebugger Ablation study results (Fig. 2).

V. Timeline

- **Weeks 1-2:** Setup local analysis infrastructure (GZoltar, SonarQube) and implement L1 agents.

Helper	RepoFocus	Summarizer	Slicer	Locator	Fixer	FixerPro	*#Plau*	Expense ($)	Level
✗	✗	✗	✗	✗	✓	✗	72	0.030	L1
✗	✗	✗	✗	✓	✓	✗	140	0.048	
✗	✗	✗	✗	✓	✓	✓	192	0.116	L2
✗	✗	✗	✓	✓	✓	✓	224	0.225	
✗	✗	✓	✓	✓	✓	✓	238	0.317	
✗	✓	✓	✓	✓	✓	✓	245	0.364	L3

Fig. 2. System Architecture

- **Weeks 3-4:** Implement L2 (Slicer/Summarizer) and the L1 $\rightarrow$ L2 transition logic.
- **Weeks 5-6:** Implement L3 (RepoFocus) and multi-agent reflection loops in FixerPro.
- **Weeks 7-8:** Evaluation on Defects4J and cost-per-bug analysis.

REFERENCES

[1] C. Lee et al., "UniDebugger: Hierarchical Multi-Agent Framework for Unified Software Debugging," arXiv:2404.17153v3, 2025.

[2] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge. In Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, SPLASH 2017, Vancouver, BC, Canada, October 23 - 27, 2017, pages 55–56. ACM.